Day 18: Networking Basics

Networking is how computers talk to each other. Every scanner, log, firewall rule, and attack you will study later depends on it.

1. IP Address and Hostname

An **IP address** identifies a device on a network, like a house address. IPv4 has four numbers (0 to 255) separated by dots: 192.168.1.10.

Type	Range	Meaning
Private	10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16	Used inside homes, labs, colleges. Not reachable from the internet
Public	Everything else	Reachable on the internet
Loopback	127.0.0.1 (localhost)	"This computer itself"

- **IPv6** is the newer format (2001:db8::1), created because IPv4 addresses are running out.
- **Subnet:** 192.168.1.0/24 means the first 24 bits are the network, so addresses .1 to .254 are devices. This is what "scan my /24 lab network" meant earlier.
- **Hostname:** a human-friendly name such as lab-pc-01 or example.com.
- **DNS** translates a hostname into an IP address, like a phone book.
- **MAC address:** a hardware address (aa:bb:cc:dd:ee:ff) used inside the local network.

Commands to try on your own PC:

bash

ipconfig # Windows: your IP (Linux/macOS: ip a or ifconfig)

hostname # your computer's name

ping 127.0.0.1 # test the network stack on your own computer

nslookup example.com # DNS lookup

2. Ports and Common Services

An IP address finds the **computer.** A **port** (0 to 65535) finds the **program** on it. Think of the IP as a building and the port as a door number.

Range	Name
0 to 1023	Well-known (standard services)
1024 to 49151	Registered
49152 to 65535	Dynamic / temporary

Port	Service	Used for	Security note

21	FTP	File transfer	Unencrypted
22	SSH	Secure remote login	Encrypted, the safe choice
23	Telnet	Remote login	**Unencrypted, avoid**
25	SMTP	Sending email	
53	DNS	Name lookup	
80	HTTP	Websites	Unencrypted
110 / 143	POP3 / IMAP	Reading email	Unencrypted versions
443	HTTPS	Secure websites	Encrypted
445	SMB	Windows file sharing	Often attacked
3306	MySQL	Database	Should not be public
3389	RDP	Windows remote desktop	Often attacked

An **IP address plus a port** is called a **socket address**: 192.168.1.10:22.

Port states:

- **Open:** a program is listening and accepts connections.
- **Closed:** nothing is listening, and the computer refuses the connection.
- **Filtered:** a firewall silently drops the traffic, so there is no answer (a timeout).

Why it matters for defence: every open port is a possible way in. A secure system keeps only the ports it needs open.

See what is listening on your own PC:

bash

netstat -an # Windows / macOS (Linux: ss -tuln)

3. TCP and HTTP Basics

TCP vs UDP

	TCP	**UDP**
Connection	Yes (handshake first)	No
Reliable and in order	Yes	No
Speed	Slower	Faster
Used for	Web, email, SSH, file transfer	DNS, video calls, games

The TCP three-way handshake

Before any data moves, TCP sets up a connection:

Client Server

| ---- SYN ----------> | "Can we talk?"

| <--- SYN-ACK --------- | "Yes, can you hear me?"

| ---- ACK ----------> | "Yes. Connected."

This handshake is why a connection to a **closed** port fails immediately (refused) and a **filtered** port just times out.

HTTP

HTTP is the language of the web, sent over TCP (port 80), or encrypted as **HTTPS** (port 443). It is plain text with a request and a response.

Request:

GET /index.html HTTP/1.1

Host: example.com

Response:

HTTP/1.1 200 OK

Content-Type: text/html

<html>...page content...</html>

Common methods	**Meaning**
GET	Fetch something
POST	Send data (forms, logins)

Status code	**Meaning**
200	OK
301 / 302	Redirect
403	Forbidden
404	Not found
500	Server error

HTTPS is HTTP wrapped in TLS encryption. Over plain HTTP, anyone on the path can read passwords and cookies.

4. Client-Server Model

Most network programs have two roles:

- **Server:** waits (**listens**) on a known port and serves requests.
- **Client:** starts the conversation by **connecting** to the server's IP and port.

Client (your browser) Server (web server)

IP 192.168.1.20, port 52814 -------> IP 192.168.1.5, port 80

<------- response

- The server uses a **fixed port** (80 for web), so clients know where to find it.
- The client gets a **temporary port** (such as 52814) assigned automatically.
- One server can handle **many clients** at once.
- The same computer can be a client and a server at the same time.

Examples: browser to web server, email app to mail server, SSH client to an SSH server, your Python script to a database.

Mini-Project: Network Facts Helper

This uses the ipaddress standard module (Day 17 idea) and a dictionary (Day 11). It does no networking. It only classifies IPs and describes ports.

python

```
# Program: Day 18 - network facts helper (practice)
import ipaddress

SERVICES = {21: "FTP", 22: "SSH", 23: "Telnet", 25: "SMTP", 53: "DNS",
            80: "HTTP", 110: "POP3", 143: "IMAP", 443: "HTTPS",
            445: "SMB", 3306: "MySQL", 3389: "RDP", 8080: "HTTP-alt"}
INSECURE = (21, 23, 25, 110, 143)


def classify_ip(text):
    try:
        ip = ipaddress.ip_address(text)
    except ValueError:
        return "Invalid IP address"
```

```
    if ip.is_loopback:
        return "Loopback (this computer only)"
    if ip.is_private:
        return "Private (inside a LAN)"
    return "Public (reachable on the internet)"



def describe_port(port):
    name = SERVICES.get(port, "unknown service")
    note = "  (unencrypted - avoid)" if port in INSECURE else ""
    return f"{port}/tcp - {name}{note}"



while True:
    cmd = input("ip / port / quit: ").strip().lower()
    if cmd == "quit":
        break
    elif cmd == "ip":
        print(classify_ip(input("IP: ").strip()))
    elif cmd == "port":
        t = input("Port: ").strip()
        if t.isdigit() and int(t) <= 65535:
            print(describe_port(int(t)))
        else:
            print("Port must be 0-65535")
```

Sample run:

```
IP: 192.168.1.5   ->  Private (inside a LAN)
IP: 8.8.8.8       ->  Public (reachable on the internet)
IP: 127.0.0.1     ->  Loopback (this computer only)
IP: 300.1.1.1     ->  Invalid IP address
Port: 23          ->  23/tcp - Telnet  (unencrypted - avoid)
```

Day 19: Python Socket Module

The socket module is Python's door to the network. Everything you learned on Day 18 (IPs, ports, TCP, clients, servers) becomes code today. It is part of the standard library, so there is nothing to install.

1. Importing socket

python

```
import socket
```

A **socket** is one end of a network connection. Your program creates a socket, and then connects, sends, and receives through it.

2. Getting Hostname and Local IP

python

```
import socket


print(socket.gethostname())               # your computer's name
print(socket.gethostbyname("localhost"))    # 127.0.0.1
print(socket.gethostbyname("example.com"))  # DNS lookup: hostname -> IP
```

Unknown hostname raises an error. Catch it (Day 16):

python

```
try:
    ip = socket.gethostbyname("no-such-host.invalid")
except socket.gaierror as e:
    print(f"Lookup failed: {e}")
```

gaierror stands for "get address info error".

Your real local (LAN) IP. gethostbyname(gethostname()) often returns 127.0.0.1 on some systems, so this trick is more reliable. It opens a UDP socket toward a made-up address and reads which local address Python chose. **No data is sent.**

python

```
def get_local_ip():
    s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    try:
        s.connect(("10.255.255.255", 1))
        return s.getsockname()[0]
    except OSError:
```

```
        return "127.0.0.1"
    finally:
        s.close()

print(get_local_ip())      # for example 192.168.1.20
```

3. Creating a TCP Socket

python

```
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

Part	**Meaning**
AF_INET	IPv4 addresses (AF_INET6 is IPv6)
SOCK_STREAM	**TCP** (SOCK_DGRAM is UDP)

Client methods:

Method	**What it does**
s.connect((host, port))	Connect to a server (note the **double brackets**: one tuple)
s.connect_ex((host, port))	Like connect, but returns 0 on success instead of raising an error
s.sendall(bytes)	Send data
s.recv(1024)	Receive up to 1024 bytes
s.close()	Close the socket

Server methods: bind((host, port)), listen(), accept().

Bytes, not strings. Networks carry bytes. Convert with encode() and decode():

python

```
data = "Hello".encode()        # b'Hello'
text = data.decode()           # 'Hello'
```

Or write a byte literal directly: b"Hello".

A tiny client-server on your own computer

Open **two terminals** in the same folder.

Terminal 1: server.py (run first)

python

```
import socket

HOST, PORT = "127.0.0.1", 5000

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server:
    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server.bind((HOST, PORT))
    server.listen()
    print(f"Server listening on {HOST}:{PORT}")

    conn, addr = server.accept()          # waits here for a client
    with conn:
        print(f"Connected by {addr[0]}:{addr[1]}")
        data = conn.recv(1024)
        print("Received:", data.decode())
        conn.sendall(b"Hello from server: " + data)
print("Server done")
```

Terminal 2: client.py

```
python
import socket

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.settimeout(3)
    s.connect(("127.0.0.1", 5000))
    s.sendall("Hi server".encode())
    reply = s.recv(1024)
    print("Reply:", reply.decode())
```

Output:

```
Server listening on 127.0.0.1:5000
Connected by 127.0.0.1:39530          <- the client's temporary port
Received: Hi server
```

Server done

and on the client:

Reply: Hello from server: Hi server

This is the Day 18 model running live: the server has the **fixed port 5000**, and the client got a **temporary port** automatically.

Binding to 127.0.0.1 means only your own computer can connect. Binding to 0.0.0.0 accepts connections from the whole network, so only do that on purpose.

4. Setting Timeout and Closing the Socket

Timeout

By default a socket can wait **forever**. A tool that hangs on one unreachable host is useless, so always set a timeout.

python

```
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.settimeout(2.0)          # seconds
```

If the time runs out, Python raises TimeoutError (older Python versions call it socket.timeout).

Common outcomes of connect():

Result	**Meaning**
Works	Port is **open**
ConnectionRefusedError	Port is **closed** (host answered "no")
TimeoutError	No answer: **filtered** by a firewall, or host is down
socket.gaierror	Hostname could not be resolved
Other OSError	Network unreachable, etc.

Closing

Always close sockets, or they leak resources and keep ports busy.

Best way: with (Day 14), which closes automatically even after an error:

python

```
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.settimeout(2)
    s.connect(("127.0.0.1", 5000))
```

```
# the socket is closed here
```

Manual way: try / finally

```
python

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
try:
    s.settimeout(2)
    s.connect(("127.0.0.1", 5000))
except OSError as e:
    print(f"Failed: {e}")
finally:
    s.close()
```

Checking one port

```
python

def check_port(host, port, timeout=1.0):
    """Return True if the TCP port accepts a connection."""
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.settimeout(timeout)
        return s.connect_ex((host, port)) == 0
```

Test it with the Python built-in web server. In one terminal run python -m http.server 8000, then:

```
python

print(check_port("127.0.0.1", 8000))     # True
print(check_port("127.0.0.1", 9))        # False
```

Your first HTTP request by hand (Day 18 meets Day 19)

With the same python -m http.server 8000 running:

```
python

import socket


with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.settimeout(3)
    s.connect(("127.0.0.1", 8000))
    s.sendall(b"GET / HTTP/1.1\r\nHost: localhost\r\nConnection: close\r\n\r\n")
```

```
    data = b""
    while True:
        chunk = s.recv(4096)
        if not chunk:                # empty = server closed the connection
            break
        data += chunk

print(data.decode(errors="replace").splitlines()[0])
```

Output: HTTP/1.0 200 OK. You just did what a browser does, in 10 lines. \r\n is the line ending HTTP requires, and the blank line at the end marks the end of the request.

Mini-Project: Own-Machine Port Checker

It checks common ports on a computer you own and reports which are open, using the Day 18 service names.

python

```
# Program: Day 19 - port checker for MY OWN machine (practice)
import socket

COMMON_PORTS = {21: "FTP", 22: "SSH", 23: "Telnet", 80: "HTTP",
                443: "HTTPS", 445: "SMB", 3306: "MySQL", 3389: "RDP",
                8000: "HTTP-dev"}


def check_port(host, port, timeout=0.5):
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.settimeout(timeout)
        return s.connect_ex((host, port)) == 0


def main():
    print("=" * 36)
    print("  OWN-MACHINE PORT CHECKER")
```

```
    print("=" * 36)

    host = input("Host [127.0.0.1]: ").strip() or "127.0.0.1"
    if host != "127.0.0.1":
        answer = input("Do you own this machine or have written permission? (yes/no): ")
        if answer.strip().lower() != "yes":
            print("Cancelled. Only test systems you are authorized to test.")
            return

    try:
        ip = socket.gethostbyname(host)
    except socket.gaierror:
        print("Could not resolve that host name.")
        return

    print(f"\nChecking {ip} ...\n")
    open_ports = []
    for port, name in COMMON_PORTS.items():
        if check_port(ip, port):
            print(f"  {port:<6}{name:<10}OPEN")
            open_ports.append(port)

    print()
    print(f"Open ports found: {len(open_ports)}")
    if 23 in open_ports:
        print("Warning: Telnet is open and unencrypted.")


main()
```

Sample run (with python -m http.server 8000 running):

```
  8000  HTTP-dev  OPEN
```

Open ports found: 1